

Vyapy QA Automation Platform

Honest project report — what it does, what it will do, and what is being built now.
Generated 18 July 2026 · Status: active development · Assessed against real Vyapy code, running.

One-line summary. An AI-assisted regression-testing platform that sits between GitHub and TestFlight for Vyapy's interconnected apps (Consumer, Business, Web). It detects pull requests, selects the relevant tests, runs them on real iOS simulators, checks whether a change in one app breaks another, and reports pass/fail back to the PR — so a broken build is caught before real users see it. This report states plainly what is working, what is blocked, and by what.

1. The problem it solves

Vyapy has four interdependent apps sharing one backend API: Staging & Production Consumer (diner) and Business (waiter/kitchen). A developer tests a feature branch in isolation — it works. It merges to `main` — the app breaks, because a change in the Business app (e.g. order status) can break the Consumer app (order tracking) through the shared API. Full manual regression after every merge is too slow to run consistently, so breakages reach TestFlight and real users.

2. What the platform actually is

It is **two systems being bridged into one**:

SYSTEM	WHAT IT IS	ROLE
Platform (this repo)	FastAPI backend + React dashboard, iOS/Appium runner, GitHub webhook, dependency graph, live simulator streaming, AI chat.	Orchestration, PR integration, reporting UI.
Vya-agentic-BOT (separate)	Android bot that drives Consumer + Business together with real cross-app synchronization and financial (bill/VAT) validation.	The mature cross-app execution engine.

These were separate projects. A bridge (built this cycle) lets the Android bot's results flow into the platform's database and dashboard, and lets the platform trigger it.

3. What works today VERIFIED

Each item below was run against real Vyapy code and observed working — not assumed.

- **iOS scenario runner.** Drives the Consumer app from plain-language steps (book a slot → open menu → add a product) with honest per-step reporting, swipe/gesture support, and a guard that refuses to tap destructive controls (logout/delete) by accident.
- **Auto-identifier catalog.** Elements with no testID are matched by on-screen text and given temporary IDs, with a confidence score and a technical-debt report telling developers exactly which screens need accessibility labels.
- **Smart PR test selection.** Diffs the *whole* PR against its merge base (not just the last commit) and selects only tests that actually exist on disk — a prior version selected phantom test files and silently ran nothing.

- **Dependency graph.** Scans Swift/JS for shared API endpoints across apps; a scanner fix now correctly links the web app and both mobile apps.
- **Android bridge.** Cross-app results (Consumer + Business per scenario) POST into the platform, apply a "both must pass" gate, and render in a Scenarios tab. Verified end-to-end.
- **Two-simulator run.** The platform can drive the iPhone (Consumer) and iPad (Business) simulators *at once*, synchronized — verified: both sims driven in parallel from a single run.
- **Supporting features:** live simulator streaming to the browser, AI chat on local Ollama, Metro auto-start, a bundle-id dropdown, and timestamped scenario recordings.

4. What we are implementing now IN PROGRESS

The current focus is a genuine, one-click **full cross-app run across both simulators**, so the Consumer books/orders/pays while the Business app accepts/cooks/bills, in sync.

PIECE	STATE
"Run Cross-App Suite" button + endpoint that boots both sims and runs them in parallel threads with sync events (book → accept → kitchen → pay)	built & verified
Business app given its <i>own</i> Metro bundler (port 8082) so it loads on the iPad instead of stalling on its splash	solved
Business login automation — dismiss the native notification alert, fill the real input fields, accept Terms & Conditions	form fills correctly
Capture the waiter → kitchen labels behind login and run the full synced journey	blocked — see §6

5. What it will do (once the blockers clear)

- On every PR: automatically run the relevant iOS tests *and* the full Consumer↔Business cross-app journey, then post a single pass/fail to the PR — blocking merges that break another app.
- Validate the money: per-guest bill, VAT, tips and totals (the Android bot already does this; it flows into the platform via the bridge).
- Explain failures: AI root-cause analysis instead of a bare "FAILED".

6. Honest limitations & what blocks a fully green run

The *mechanism* for a full cross-app run is built and verified. It does not yet complete end-to-end, for reasons that are **in the Vyapy apps and accounts, not the platform**:

BLOCKER	WHERE	OWNER
Consumer crash <code>TextEncoder</code> is undefined — the QR code on every order screen crashes the app after booking	qrcode lib via <code>UpcomingCard.js:2789</code>	Vyapy app (needs a polyfill)
Consumer crash <code>appointment.type</code> null on the booking popup	<code>getReservationParticipantCount</code>	Vyapy app

Business login	Sign-in rejects with an unhandled promise — also fails when signed in <i>by hand</i> , so it is the account/backend, not the automation	Business auth on <code>api.vyapy.com</code>	Vyapy backend / account
Metro memory	the bundler runs out of memory on large builds	tooling	Mitigated (larger heap); not root-fixed

What this means plainly: the platform can drive both simulators together, log the Business form in, and record synced results. It cannot yet show a green Consumer checkout (two app crashes) or reach the waiter/kitchen screens (the Business login is rejected at the account level, confirmed by a manual sign-in also failing). Those are app- and account-side fixes.

7. Proof of value — real bugs this platform found

Beyond "it runs tests", the platform has already surfaced genuine defects in live Vyapy code:

- The `TextEncoder` QR-code crash on order screens (blocks checkout).
- The `appointment.type` null crash on the booking popup.
- The Business app cannot bundle alongside the Consumer app without its own packager.
- A native notification permission alert silently blocking the Business login form.

None of these were known before the platform drove the apps. Catching them before TestFlight is exactly the platform's purpose.

8. Technology

LAYER	TECH	LAYER	TECH
Backend	Python / FastAPI / SQLite	Dependency analysis	Graphify + custom HTTP endpoint scanner
Frontend	React + TypeScript + Vite	AI / LLM	Ollama llama3.2 (local, free)
Mobile automation	Appium + XCUITest + WebDriverAgent	Live streaming	WebSocket + xcrun simctl
Cross-app engine	Android bot (threading sync) + two-sim iOS orchestrator	CI trigger	GitHub webhook

This report is written to be accurate, not promotional. "Verified" items were observed running against real Vyapy code; "blocked" items are stated with their cause and owner. The platform's job is to catch cross-app regressions before TestFlight; it demonstrably finds real bugs today, and the remaining gaps to a fully automated green run are app- and account-side, now clearly identified.